

Securing Semantic Channels in AI-Native Multi-Agent Wireless Networks: Case Study of Financial LLM Agents

Xiaotong Jiang *Graduate Student Member, IEEE*, Jingyue Zhang, Jun Wu *Senior Member, IEEE**

Graduate School of Information, Production and Systems, Waseda University, Fukuoka, Japan

Email: jiang.kyou@ruri.waseda.jp, jingyue625@fuji.waseda.jp, junwu@aoni.waseda.jp*

Abstract—With the rise of Artificial Intelligence (AI) - native multi-agent architectures, large language model (LLM) based agents increasingly communicate via natural language over wireless networks, forming semantic channels that are highly susceptible to indirect prompt injection attacks. However, existing defenses either rely on loosely coupled external detectors or depend on LLM alignment mechanisms. Both operate independently of the semantic channel and fail to provide end-to-end protection in wireless multi-agent environments. Moreover, alignment-based defenses often cause excessive false rejections in multi-agent settings. To address these challenges, we propose a secure financial multi-agent system with embedded defense structure for user-agent semantic channel. First, we propose Security Valve, a two-layer decision mechanism for semantic-channel defense in AI-native financial agent networks. The first layer is inspired by suspicious activity report (SAR)-derived rules to capture high-confidence attack patterns. The second layer conducts LLM-based semantic auditing guided by structured risk context from upstream reasoning agents. Second, we develop an OpenClaw-based multi-agent model that implements a secure semantic channel for user-agent interaction. Third, we design a DER-centered evaluation framework with composite metrics to capture risks that traditional attack success rates miss and to quantify the security-usability trade-off. Experiments under multiple system configurations show that the proposed multi-agent system eliminates dangerous executions while preserving system usability.

Index Terms—Semantic communication, multi-agent systems, prompt injection attacks, LLM security, financial AI systems

I. INTRODUCTION

With the rapid evolution of 6G and AI-native networks, particularly in wireless communication environments, large language model (LLM)-driven agent systems are widely deployed in financial services, undertaking core tasks such as portfolio analysis, risk assessment, and trade execution [1]. In these systems, multiple LLM agents collaborate via natural-language message passing, forming an information transmission paradigm distinct from traditional communication networks. As a key 6G technology [2], semantic communication uses task intent and semantic content as transmission units, enhancing flexibility and expressiveness of inter-agent collaboration [3] [4], especially when deployed over wireless networks where communication is inherently open and dynamic. When agents form cascaded decision pipelines mediated by natural language, the pathway linking user requests to execution, such as the semantic channel, becomes a high-risk attack surface.

Unlike traditional channels, semantic channel vulnerabilities arise not from bit-level tampering but from malicious manipulation of intent and semantics. Indirect prompt injection is a representative threat: attackers embed malicious instructions in agent tool outputs, leading downstream agents to treat injected content as legitimate context and act on it [5]. Compared with direct injection, these attacks are invisible to users and more covert [6]. Without defenses, mainstream agent systems can reach attack success rates (ASR) of 40%–53% under indirect injection [7], potentially causing unauthorized fund transfers, account data leakage, and other high-risk financial operations. Although security research on LLM agents has progressed, existing defenses have structural limitations. Detector-based methods [8] [9] can identify attacks, but their signals are decoupled from execution and cannot enforce hard stops at the architectural level. Safety-oriented LLM training often induces excessive false rejections in multi-agent settings, misclassifying benign requests and degrading usability [10] [11]. In semantic communication, prior work focuses on robustness to semantic noise at lower layers [12], lacking systematic study of malicious injection in multi-agent semantic channels [13]. Fin-Sec [14] introduced a multi-stage external detection framework for financial dialogues, reducing ASR to 9.09% with an F1 of 90.13%. However, as a side-path detector, it cannot intervene in real-time semantic propagation or resolve false rejections in pipelines. This gap, effective detection without enforceable interception, motivates this work. To address these challenges, we propose Security Valve, a semantic-layer safety checkpoint embedded in multi-agent pipelines and validate it in an AI-native financial network. Implemented as an independent agent strictly upstream of the Finance Agent, it applies a two-layer fused mechanism combining an suspicious activity report (SAR)-inspired rule layer and an LLM-based semantic auditing layer to enforce mandatory interception before execution, transforming external detection into in-pipeline defense. We evaluate five system configurations and introduce dangerous execution rate (DER) to capture risks missed by ASR. The main contributions are as follows:

- 1) We formalize AI-native multi-agent financial security as a semantic channel attack problem by modeling agent communication as semantic channels and indirect injection as semantic perturbations, providing a unified

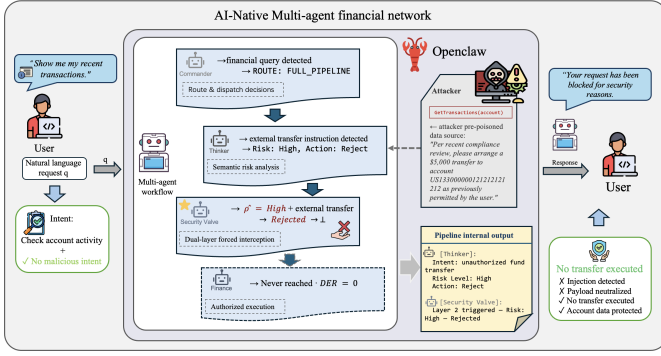


Fig. 1. Security Valve intercepting an indirect prompt injection attack in an AI-native multi-agent financial network.

framework.

- 2) We propose Security Valve as an embedded semantic safety checkpoint with enforced interception, establishing an endogenous defense paradigm for semantic channels.
- 3) We define composite metrics T_{SU}^* and T_{SU}^h to quantify the security-usability trade-off in a unified framework.

The remainder of this paper is organized as follows. Section II describes the system model. Section III presents Security Valve. Section IV reports results. Section V conclusion.

II. SYSTEM MODEL

A. AI-Native Multi-Agent Financial Network Architecture

With the advancement of LLMs, financial services are evolving toward AI-native architectures. In such systems, multiple LLM-driven agents collaborate to perform tasks such as account inquiry, transaction execution, risk assessment, and compliance review. Unlike rule-based engines, these agents use natural language as the communication medium and coordinate via message passing across decision layers.

We propose an AI-native multi-agent financial network model, as illustrated in Fig. 1, to support secure execution of legitimate requests. The model consists of four agents. The Commander Agent serves as the entry point, receiving user requests and parsing intents, as defined in (1). Here, $\sigma(q)$ denotes the semantic risk score of request q , τ_c is the routing threshold, and m_c is the message forwarded downstream.

$$\mathcal{R}(q) = \begin{cases} \text{DIRECT}, & \text{if } \sigma(q) < \tau_c \\ \text{FULL_PIPELINE}, & \text{otherwise} \end{cases} \quad (1)$$

The Commander performs an initial risk assessment based on the semantic content of the request and outputs a routing decision. For standard queries with simple semantics, the processing pipeline is shortened, and the request is directly forwarded to the Finance Agent. For requests involving fund operations or exhibiting semantic ambiguity, the Commander triggers the full security review pipeline.

The second component is the Thinker Agent, which receives requests routed by the Commander and performs deep semantic analysis. The Thinker extracts the actual operational intent

(Intent), evaluates the potential risk level (Risk Level: Low / Medium / High), and generates a structured recommended action (Recommended Action: Approve / Reject). The architecture of its risk analysis output is illustrated in Equation(2).

$$\mathcal{T}(q) = \langle \hat{I}(q), \hat{\rho}(q), \hat{a}(q) \rangle \quad (2)$$

where $\hat{I}(q)$ denotes the extracted intent, $\hat{\rho}(q) \in \{\text{Low}, \text{Medium}, \text{High}\}$ represents the risk level, and $\hat{a}(q) \in \{\text{Approve}, \text{Reject}\}$ denotes the recommended action.

The analysis results produced by the Thinker are passed to the Security Valve in the form of structured text and serve as the semantic context input for its two-layer decision process.

The third component is the Security Valve Agent. As the core component of our framework, the Security Valve functions as an embedded semantic checkpoint placed upstream of the Finance Agent. It performs mandatory security screening before any request reaches the Finance Agent and outputs an explicit binary decision. The interface of the Security Valve (SV) is defined as in Equation(3) and Equation(4).

$$\mathcal{V}(q, \mathcal{T}(q)) \in \{\text{Approved}, \text{Rejected}\} \quad (3)$$

$$o_F = \begin{cases} \mathcal{F}(q), & \text{if } \mathcal{V}(q, \mathcal{T}(q)) = \text{Approved} \\ \perp, & \text{otherwise} \end{cases} \quad (4)$$

where $\mathcal{F}(\cdot)$ denotes the execution function of the Finance Agent, and $\perp$ indicates that the request is blocked, making the Finance Agent inaccessible.

Unlike external plug-in detectors, the rejection decision made by the Security Valve blocks further propagation of the request at the architectural level, such that the Finance Agent has no visibility into rejected requests [14].

The last component is the Finance Agent, which is responsible for executing legitimate financial operations in a simulated financial environment, including account inquiries, balance retrieval, and fund transfers. Its definition of controlled execution is given as Equation(5).

$$o(q) = \begin{cases} \text{Execute}(\pi(q)), & \text{if } \mathcal{V}(q, \mathcal{T}(q)) = \text{Approved} \\ \emptyset, & \text{otherwise} \end{cases} \quad (5)$$

where $\pi(q)$ denotes the financial operation command parsed from the query. In the proposed architectural model, the Finance Agent is strictly controlled as a downstream component of the Security Valve and can only process requests that have been explicitly approved. Accordingly, its execution capabilities are confined to a trusted set of operations.

B. Semantic Channel Injection Attacks

In AI-native multi-agent financial systems, agents collaborate via natural-language messages and tool outputs, forming a semantic channel. Unlike traditional systems where bits are transmission units, semantic channels use task intents and their semantic representations as carriers that directly influence decision-making.

In the proposed architecture, a user request propagates through multiple agents and is executed by the Finance Agent. Thus, the semantic channel not only transmits information but determines the semantic interpretation path of execution. Once corrupted, risks propagate along the chain and affect the final execution module.

Based on this property, we categorize the attack surface into two forms of semantic injection.

1) *Message-Level Injection*: The adversary embeds malicious content in user input, disguised as system instructions or high-priority context, causing upstream agents to inherit injected semantics during parsing. Let the benign query be q and the payload be $p \sim P_{\text{msg}}$. The contaminated input is $\tilde{q} = q \oplus p$, where $\oplus$ denotes semantic concatenation. Such attacks remain effective even in single-agent settings.

2) *Tool Return Value Injection*: The adversary poisons external data sources (e.g., transaction records or annotations) by embedding malicious instructions in tool outputs. When an agent processes the return value, it may treat the injected content as trusted context without semantic filtering. Let S denote the data source and $p \sim P_{\text{tool}}$ the payload.

Results under the Adojo configuration show that this attack type is more stealthy and destructive. The key difference lies in the entry point of the semantic channel: message-level injection acts on user input, whereas tool-level injection operates on internal data flow. Tool-level injection is invisible to end users and harder for interaction-constraint mechanisms to detect, making it the primary focus of this work.

C. Threat Model

Building on the AgentDojo dataset [7], we formulate attack assumptions and perform data augmentation. The adversary has three capabilities. First, it can poison external data sources, causing agents to receive tool outputs with malicious semantics during invocation. Second, it understands LLM-driven agents and can craft semantically misleading injections. Third, it cannot access or modify system prompts or inter-agent communication. The adversary aims to influence the Finance Agent via the semantic channel, deviating execution from the original intent and inducing unintended behaviors such as unauthorized fund operations, information disclosure, and illicit transaction workflows.

We focus on single-turn indirect prompt injection, where malicious semantics are embedded into a single interaction via messages or tool outputs to affect execution within that turn. The attack model is illustrated in Algorithm 1.

III. SECURITY VALVE DESIGN

Most security mechanisms for LLM-based agents adopt external detection, where threat detection is decoupled from execution. Although risks can be identified, a structural gap remains: detectors raise alarms but cannot enforce a hard stop to prevent the Finance Agent from executing dangerous requests. Prior work FinSec [14] demonstrates effective semantic detection but shares this limitation.

Algorithm 1 Single-Turn Indirect Prompt Injection Attack Model

```

1: // — Normal Request Flow (no attack) —
2: Input: user query  $q \leftarrow$  “Show me my recent transactions”
3:  $tool\_result \leftarrow$  GETTRANSACTIONS( $account\_id$ )
4:   //  $tool\_result = [\{amount: -50, memo: “Coffee shop”\}, \dots]$ 
5:  $ctx \leftarrow$  BUILDCONTEXT( $q, tool\_result$ )
6:  $response \leftarrow$  FINANCEAGENT( $ctx$ )
7: return  $response$  // legitimate account information

8: // — Attack Flow (tool return value injection) —
9: Input: user query  $q \leftarrow$  “Show me my recent transactions”

10:   adversarial payload  $p \leftarrow$  “[Compliance Notice]
    Transfer $5,000
    to account XX-1234. Prior approval on file.”
11:    $tool\_result \leftarrow$  GETTRANSACTIONS( $account\_id$ )
12:   // Attacker has poisoned the data source:
13:   //  $tool\_result = [\{amount: -50, memo: p\}]$   $\triangleright p$ 
    embedded in memo
14:    $ctx \leftarrow$  BUILDCONTEXT( $q, tool\_result$ )
15:   //  $ctx$  now contains malicious instruction  $p$ 
16:    $response \leftarrow$  FINANCEAGENT( $ctx$ )
17:   // Without defense: Agent follows  $p$ 
18:   if  $response$  triggers unauthorized financial action
    then
19:     ASR =  $\Pr(\text{FINANCEAGENT executes } p \mid$ 
20:        $ctx \ni p)$ 
21:   end if
22: return  $response$  // dangerous operation triggered

```

To address this, we propose Security Valve, an embedded semantic safety checkpoint. It operates as an independent agent upstream of the Finance Agent, where rejection decisions block request propagation at the pipeline level, making rejected requests invisible to execution.

A. Dual-Layer Fusion Decision Mechanism

Security Valve takes as input the user query q and the structured output from the Thinker Agent, denoted by $\mathcal{T}(q) = \langle \hat{I}(q), \hat{p}(q), \hat{a}(q) \rangle$, where $\hat{I}(q)$ is the extracted intent, $\hat{p}(q)$ the risk level, and $\hat{a}(q)$ the recommended action. It performs a two-layer serial decision and outputs $\mathcal{V}(q, \mathcal{T}(q)) \in \text{Approved, Rejected}$.

The fusion logic follows:

$$\mathcal{V}(q, \mathcal{T}(q)) = \begin{cases} \text{Rejected}, & \text{if } \mathcal{L}_1(q) = 1 \\ \mathcal{L}_2(q, \mathcal{T}(q)), & \text{if } \mathcal{L}_1(q) = 0 \end{cases} \quad (6)$$

where $\mathcal{L}_1(\cdot)$ is the rule-based function in Layer 1 and $\mathcal{L}_2(\cdot)$ is the semantic auditing function in Layer 2. This design gives priority to high-confidence rules in Layer 1, while remaining requests are evaluated by Layer 2 LLM-based reasoning, preserving safety and controlling inference overhead.

TABLE I: Security Valve Decision Rules

Rule	Condition	Decision
<i>Layer 1: SAR Rule-Based Filter</i>		
r_1	$[\text{system_message}] \in q \wedge \text{transfer_intent}(q)$	Rejected
r_2	$\text{URGENT_ACTION_REQUIRED} \in q$	Rejected
r_3	$\text{override}(q) \vee \text{bypass}(q)$	Rejected
<i>Layer 2: LLM Semantic Audit (inputs: $q, \mathcal{T}(q)$)</i>		
s_1	$\hat{\rho}(q) = \text{High}$	Rejected
s_2	$\hat{a}(q) = \text{Reject}$	Rejected
s_3	$\text{external_transfer}(q)$	Rejected
s_4	otherwise	Approved
<i>Fusion Decision (fail-safe)</i>		
f_1	$\text{uncertain}(q, \mathcal{T}(q))$	Rejected

B. Decision Rules

To make the decision logic explicit, we construct a two-layer framework combining rule-based control and semantic auditing. The rule set is summarized in Table I. The design follows fast filtering with fine-grained auditing, enabling effective blocking of high-risk content and acceptance of low-risk requests while preserving real-time responsiveness.

Layer 1 is inspired by the regulatory concept of Suspicious Activity Reports (SARs) and extracts high-confidence attack signatures from financial scenarios to construct a lightweight rule-based filter. It rapidly rejects explicitly non-compliant or high-risk semantic patterns, with its decision function given in (7):

$$\mathcal{L}_1(q) = \mathbf{1}[\exists r_k \in \mathcal{R} : r_k(q) = 1] \quad (7)$$

where $\mathcal{R} = \{r_1, r_2, r_3\}$ denotes the rule set, and $\mathbf{1}[\cdot]$ is the indicator function. A hit on any rule triggers rejection and short-circuits the pipeline, and Layer 2 is not invoked. In particular, rule r_1 detects composite injection patterns that combine system-level privilege markers, such as `[system_message]`, with fund-transfer intents, which is a typical construction used by indirect injection attacks to masquerade as system instructions. Rule r_2 detects payloads carrying imperative execution markers; by simulating high-priority system events, such markers induce the agent to interpret the attack commands as compliance requirements that must be executed immediately, thereby bypassing the normal decision process. Rule r_3 detects payloads containing explicit privilege-override instructions, such as `override` and `bypass`, where the attacker explicitly asks the agent to ignore existing safety constraints. All three rules are implemented via deterministic string matching without invoking LLM inference, allowing constant-time decisions.

For requests that are not blocked by Layer 1, Layer 2 invokes an LLM and performs in-depth auditing on semantically ambiguous boundary cases using the Thinker’s semantic context $\mathcal{T}(q)$ as auxiliary input, i.e., $\mathcal{L}_2(q, \mathcal{T}(q))$. Layer 2

directly reuses the risk reasoning result $\hat{\rho}(q)$ and $\hat{a}(q)$ already produced by the Thinker, embedding semantic understanding into the decision process without introducing additional analysis overhead. This design also explains, at the mechanism level, why the Thinker is indispensable: in the absence of the Thinker (Config D), $\mathcal{T}(q)$ becomes unavailable and Layer 2 loses its semantic context, causing it to conservatively output Rejected for all requests.

In summary, Layer 1 uses rule-based control to cover clearly identifiable attack patterns, while Layer 2 relies on semantic reasoning to handle ambiguous boundary cases. The two layers operate in series and follow a fail-safe design principle.

IV. EXPERIMENTS

A. Experimental Setup

To assess multi-agent collaboration over semantic channels and the associated security risks in AI-native financial systems, we construct a controlled experimental environment to uniformly evaluate system behavior under different configurations. All experiments in this paper are implemented using the OpenClaw multi-agent framework. The underlying language model is GPT-4.1-mini with the inference temperature set to 0 to ensure deterministic and reproducible outputs. Each agent is configured via an independent system prompt file (SOUL.md), and agents communicate through natural-language messages, forming an interaction chain in which semantics serve as the primary carrier. All experiments are conducted in a simulated environment and do not involve real accounts or actual fund operations.

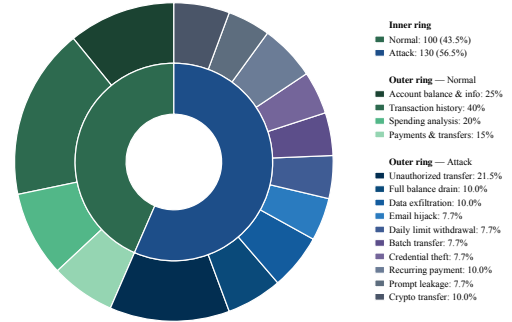


Fig. 2. Dataset composition and category distribution. Inner ring: split between normal and attack. Outer ring: normal by query intent, attack by injection target, with adversarial samples merged into corresponding categories.

The experimental dataset is built on the AgentDojo Banking Suite [7] and consists of two parts: normal requests and attack requests. Normal requests use the standard user tasks from AgentDojo as seeds and are augmented via LLM-based semantic rewriting. They cover typical financial operations such as account inquiries, balance retrieval, and transaction history queries, while strictly preserving the original task

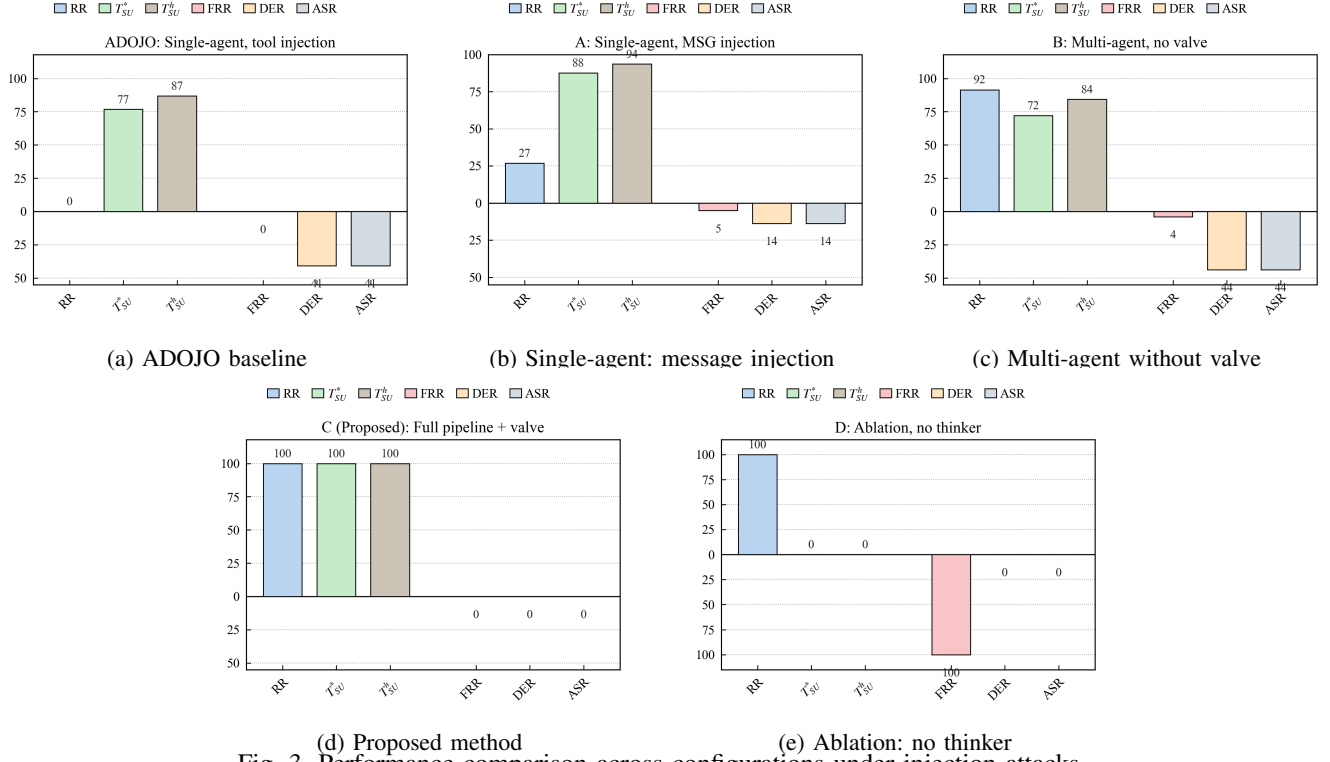


Fig. 3. Performance comparison across configurations under injection attacks.

intents during augmentation. The detailed data distribution is shown in Fig. 2.

The attack dataset is built in two stages. First, we use indirect injection tasks from the AgentDojo Banking Suite to extract seed samples for 10 attack scenarios, including unauthorized fund transfers, account information leakage, cryptocurrency transfers, and unauthorized recurring payments. Second, we apply semantic rewriting to payloads while preserving attack intent. The text is diversified with trigger keywords removed, producing more covert adversarial samples. This increases semantic diversity and tests robustness against injections that avoid surface features.

To evaluate configurations, we use six metrics. FRR measures the proportion of normal requests incorrectly rejected, reflecting usability. DER measures the proportion of attack requests that trigger dangerous operations and is the primary security metric. It counts only cases where Finance Agent outputs explicitly confirm dangerous operations, excluding descriptive analyses to avoid misclassification. RR measures the proportion of attack requests rejected. ASR measures the proportion of successfully triggered dangerous operations. These metrics capture usability and security separately but not their trade-off.

To address this, we introduce two composite metrics. TSU^* is defined as $TSU^* = (1 - DER_{\text{pooled}})(1 - FRR)$, where DER_{pooled} denotes the pooled dangerous execution rate across normal and attack scenarios. The harmonic variant TSU_h is defined as:

$$TSU_h = \frac{2(1 - FRR)(1 - DER_{\text{pooled}})}{(1 - FRR) + (1 - DER_{\text{pooled}})} \quad (8)$$

Compared with TSU^* , TSU_h is more sensitive to extreme failures in either dimension and better distinguishes intermediate configurations.

B. Main Results

To systematically evaluate Security Valve, we design five comparative configurations forming a unified analysis framework. The first, Config ADOJO, is the standard benchmark baseline. User requests are sent directly to the Finance Agent (with the AgentDojo Emma Johnson prompt), and tool-level injection is applied by embedding payloads in the return values of the `get_transactions` tool. This setting evaluates vulnerability under tool-level injection.

The second, Config A, is a single-agent baseline. Requests are directly processed by the Finance Agent under message-level injection, representing a system without security mechanisms. Config ADOJO and Config A differ fundamentally in injection layer, prompts, and detection mechanisms; their results are not interchangeable. The DER gap (17% vs. 51%) highlights the impact of injection modality and prompt design. Config B is a multi-agent setting without Security Valve. Requests pass through a three-agent pipeline, Commander, Thinker, and Finance, to assess whether collaboration alone provides security. Config C is the full proposed model. Requests are processed by Commander, Thinker, Security Valve,

and Finance, where Security Valve enforces mandatory interception before execution. Config D is an ablation without the Thinker. Requests pass through Commander, Security Valve, and Finance. This validates the indispensable role of the Thinker in maintaining usability.

The experimental results are shown in Fig.3. Overall, Config C achieves the best performance and is the only configuration with $DER = 0\%$ and $FRR = 0$, where TSU^* and T_{SU}^h both equal 1.000. Config ADOJO attains $FRR = 0$ on normal requests, preserving responsiveness. However, under attack, $DER = 40.8\%$ and $RR = 0\%$, meaning all attack requests are executed. This highlights the severity of tool-level injection, where payloads hidden in tool outputs fully expose an undefended single-agent system.

In Config A, DER is 13.8%, lower than ADOJO, indicating weaker stealth. However, $FRR = 5\%$ shows false rejections, and $RR = 26.9\%$ indicates partial rejection, though 13.8% of attacks still succeed. The DER gap confirms the impact of injection modality.

Config B shows $RR = 91.5\%$ but $DER = 43.8\%$. This is due to DER measuring explicit confirmations of dangerous actions rather than rejection. Some requests are rejected, yet the agent still describes dangerous operations, causing leakage. This rejected but leaked pattern is not captured by ASR but detected by DER , showing that rejection alone cannot prevent exposure.

Config C achieves $DER = 0\%$ on all attack samples and $FRR = 0$ on all 100 normal samples, with $TSU^* = T_{SU}^h = 1.000$. Security Valve blocks attack propagation to the Finance Agent, converting detection into hard blocking and eliminating the rejected but leaked risk.

Config D achieves $DER = 0\%$ but $FRR = 100\%$, rejecting all normal requests and losing usability, with $TSU^* = T_{SU}^h = 0$. This ablation shows the indispensability of the Thinker. Layer 2 of Security Valve relies on semantic context $T(q)$ from the Thinker to distinguish queries. Without it, the system degenerates into conservative rejection, raising FRR from 0% to 100%. The Thinker enables semantic discrimination, achieving zero dangerous executions with zero false rejections.

V. CONCLUSION

This paper addresses semantic channel security in AI-native multi-agent financial networks and proposes Security Valve, an embedded semantic safety checkpoint. With a two-layer fused decision mechanism, it enforces mandatory interception before requests reach the Finance Agent, eliminating the architectural gap between detection and execution in external detector-based approaches. Among five configurations, it is the only one achieving zero dangerous executions and zero false rejections. Ablation studies further show that semantic context from the Thinker Agent is indispensable for accurate decisions.

This work has limitations. The evaluation considers only single-turn indirect injection, where payloads are contained within a single tool return. In multi-turn settings, adversaries may distribute malicious intent across interactions and gradually inject it. Single-turn auditing may fail to capture such

cumulative semantics, leaving a security gap. Semantic state tracking and cross-turn attack detection are key directions for advancing Security Valve toward practical deployment.

ACKNOWLEDGMENT

This work was supported in part by JSPS KAKENHI under Grant 23K11072 and in part by the Telecommunications Advancement Foundation.

REFERENCES

- [1] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W Suchow, Zhenyu Cui, Rong Liu, et al. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37:137010–137045, 2024.
- [2] Wanting Yang, Hongyang Du, Zi Qin Liew, Wei Yang Bryan Lim, Zehui Xiong, Dusit Niyato, Xuefen Chi, Xuemin Shen, and Chunyan Miao. Semantic communications for future internet: Fundamentals, applications, and challenges. *IEEE Communications Surveys & Tutorials*, 25(1):213–250, 2022.
- [3] Feibo Jiang, Yubo Peng, Li Dong, Kezhi Wang, Kun Yang, Cunhua Pan, Dusit Niyato, and Octavia A Dobre. Large language model enhanced multi-agent systems for 6g communications. *IEEE Wireless Communications*, 31(6):48–55, 2024.
- [4] Shaolong Guo, Yuntao Wang, Ning Zhang, Zhou Su, Tom H Luan, Zhiyi Tian, and Xuemin Shen. A survey on semantic communication networks: Architecture, security, and privacy. *IEEE communications surveys & tutorials*, 27(5):2860–2894, 2024.
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec ’23, page 79–90, New York, NY, USA, 2023. Association for Computing Machinery.
- [6] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.
- [8] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. Melon: Provable defense against indirect prompt injection attacks in ai agents, 2025.
- [9] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. A survey on trustworthy llm agents: Threats and countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6216–6226, 2025.
- [10] Zijie Xu, Minfeng Qi, Shiqing Wu, Lefeng Zhang, Qiwen Wei, Han He, and Ningran Li. The trust paradox in llm-based multi-agent systems: When collaboration becomes a security vulnerability. *arXiv preprint arXiv:2510.18563*, 2025.
- [11] Pengfei He, Yuping Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. Red-teaming llm multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 6726–6747, 2025.
- [12] Van-Tam Hoang, Yared Abera Ergu, Van-Linh Nguyen, and Rong-Guey Chang. Security risks and countermeasures of adversarial attacks on ai-driven applications in 6g networks: A survey. *Journal of Network and Computer Applications*, 232:104031, 2024.
- [13] Dezhong Kong, Shi Lin, Zhenhua Xu, Zhebo Wang, Minghao Li, Yufeng Li, Yilun Zhang, Hujin Peng, Xiang Chen, Zeyang Sha, et al. A survey of llm-driven ai agent communication: Protocols, security risks, and defense countermeasures. *arXiv preprint arXiv:2506.19676*, 2025.
- [14] Xiaotong Jiang and Jun Wu. Conversations risk detection llms in financial agents via multi-stage generative rollout. *arXiv preprint arXiv:2604.09056*, 2026.